Day 66: The GRU — a Simpler Gated Cell, From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 4 | Complete study guide

Watch: Simple Explanation of GRU (Gated Recurrent Units) | Deep Learning Tutorial 37 (codebasics) | Gated Recurrent Unit (GRU) Equations Explained (DataMListic)

Script: `day66_gru_from_scratch.py` · every number in this guide is copied by program from `day66_run_output.txt`, the real output of that script.

The GRU keeps the LSTM's central idea, a gate that decides how much of the old memory to keep, and drops the rest of the machinery: one state instead of two, and three weight blocks instead of four. Today it is built and checked exactly as the LSTM was over Days 63–65, then trained on Day 65's grid so the two tables sit side by side. On that grid the GRU learned fewer runs (0, 3 and 6 of 18 at keep-bias 0, 1, 2, against the LSTM's 0, 10 and 16). A follow-up separates “slower” from “unable” and finds the one place the GRU's simpler design really costs it.

0. Where today fits

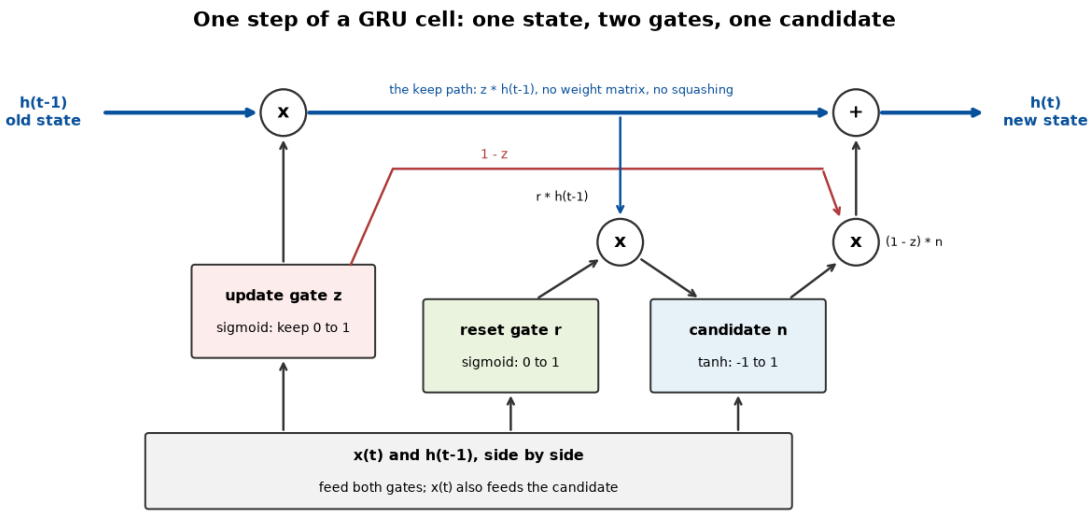
DEFINITION — start here, no prior knowledge assumed

The **GRU** (Gated Recurrent Unit, Cho et al., 2014) is a recurrent cell with **one** state h and two gates. The **update gate z** decides how much of the old state to keep; whatever it does not keep is filled with a **candidate** n. The **reset gate r** decides how much of the old state the candidate is allowed to look at.

REAL-WORLD SCENARIO

The LSTM is a whiteboard with a separate notebook: you wipe, write and read out as three separate decisions. The GRU is a single whiteboard with one rule: whatever share of the board you do not keep gets written over with new notes. Simpler, one fewer decision, but keeping more always means writing less.

SYSTEM DIAGRAM



One step of a GRU. The blue line is the keep path (z times the old state). 1 - z, in red, decides how much of the candidate n is written. The reset gate scales the old state before the candidate sees it.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One step of a GRU cell

DEFINITION — start here, no prior knowledge assumed

A GRU step reads this step's input x and the old state h, and returns the new state. In between: two gates from [x, h], a candidate from x and the *reset* old state r · h, and a blend of old state and candidate controlled by z.

REAL-WORLD SCENARIO

Updating a running summary of a long meeting. For each new remark you decide how much of the summary to keep as it is (update gate). When you draft the new sentence, you may choose to ignore what the summary already says, for example when the topic changes (reset gate).

MATHEMATICS

$$z = \sigma([x, h] W_z + b_z) \quad \text{update gate: share of the old state to keep}$$

$$r = \sigma([x, h] W_r + b_r) \quad \text{reset gate}$$

$$n = \tanh(x W_{xn} + (r \cdot h) W_{hn} + b_n) \quad \text{candidate}$$

$$h_{new} = z \cdot h + (1 - z) \cdot n$$

$$\text{weights: GRU } 3(D + H + 1)H \quad \text{LSTM } 4(D + H + 1)H \quad \text{RNN } (D + H + 1)H$$

READ THIS FIRST

Conventions differ. Some papers and libraries write $h_{new} = (1 - z) \cdot h + z \cdot n$, so their z means “how much to replace”. This series follows Cho et al. and PyTorch: z multiplies the old state, so z is the keep share and a positive bias on z means “keep”, like the LSTM's forget bias.

SPECIFICATION

Hand trace	One input, one hidden unit, x = 1.0, old h = 0.5, hand-chosen weights.
Random check	5 examples, 3 inputs, 4 hidden units, random biases; the function against the formulas written gate by gate.
Weights	Counted for three sizes against the RNN and the LSTM.

Why this test

The candidate uses r · h, not h, so its hidden weights cannot be stacked with the gates' in one product the way the LSTM's were. That split is where an implementation is most likely to go wrong.

IMPLEMENTATION

```
def gru_step_forward(x_t, h_prev, W, b):
    D, H = x_t.shape[1], h_prev.shape[1]
    zr = np.concatenate([x_t, h_prev], axis=1) @ W[:, :2 * H] + b[:2 * H]
    z = sigmoid(zr[:, :H])
    r = sigmoid(zr[:, H:])
    n = np.tanh(x_t @ W[:D, 2 * H:] + (r * h_prev) @ W[D:, 2 * H:] + b[2 * H:])
    h = z * h_prev + (1.0 - z) * n
    return h, (z, r, n)
```

ACTUAL OUTPUT

```
inputs: x = 1.0, old hidden state h = 0.5
quantity                      by hand   function
update gate z (sigmoid)        0.7503     0.7503
reset gate r (sigmoid)         0.3100     0.3100
candidate n (tanh)             0.9227     0.9227
new hidden state h             0.6056     0.6056
  new h = z * old h + (1 - z) * n = 0.7503 * 0.5 + 0.2497 * (0.9227) = 0.6056
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
random check (5 examples, 3 inputs, 4 hidden units): max difference from the gate-by-gate formulas
    0.00e+00
 input  hidden  RNN weights  GRU weights  LSTM weights
     1      16          288          864          1152
    32      32         2080         6240          8320
    64     128        24704        74112         98816
Actual output from this lesson's run.
```

What it means

The function matches the hand arithmetic on every line and the gate-by-gate formulas exactly (0.00e+00). A GRU has three quarters of an LSTM's weights: 6240 against 8320 at Day 62's tagger size.

WHY CHOOSE THIS?

One state and three weight blocks: a quarter fewer weights than an LSTM and a shorter backward pass.

WHY NOT THIS? (TRADEOFFS)

Keeping and writing are tied to one gate, z and 1 - z. Section 5b shows when that matters.

2. Forward and backward over sequences, gradient-checked

DEFINITION — start here, no prior knowledge assumed

gru_forward loops the step over time with Day 64's padding rule (one state, so one carry line) and caches the gates. **gru_backward** walks the cache backwards. The gradient for the old state arrives by three routes: straight through z, through the reset gate into the candidate, and through the two gates.

REAL-WORLD SCENARIO

Same audit as Day 65, simpler ledger: one column of running totals instead of two, so fewer places for a mistake to hide, but every entry still has to be checked.

MATHEMATICS

$dz = dh \cdot (h_{prev} - n)$, $dn = dh \cdot (1 - z)$, $\mathbf{dh_{prev} \leftarrow dh \cdot z}$ *(the direct route)*

$da_n = dn\,(1 - n^2)$; $d(r{\cdot}h) = da_n\, W_{hn}{}^T$; $dr = d(r{\cdot}h) \cdot h_{prev}$; $dh_{prev} \mathrel{+}= d(r{\cdot}h) \cdot r$

gates: $[da_z, da_r]$ *through their sigmoids, then dW, dx and* dh_{prev} *through* $[x, h]$

SPECIFICATION

(a) On its own	4 padded sequences (lengths 5, 2, 4, 1), 2 inputs, 3 hidden units, random weights and biases, random h0. L = sum(R1 · every h_t) + sum(R2 · last h). Checked: W, b, X, h0.
(b) Through the classifier	GRUClassifier's mean cross-entropy; checked: W, b, Why, by.
Pass rule	Day 62's, for every entry.

Why this test

Day 65's rule: check the function with a loss that uses every output, then check the exact path training uses.

IMPLEMENTATION

```
def gru_backward(dH, dh_last, cache):
    X, h0, W, mask = cache["X"], cache["h0"], cache["W"], cache["mask"]
    Hs, Z, R, N = cache["H"], cache["Z"], cache["R"], cache["N"]
    m, T, D = X.shape
    H = h0.shape[1]
    dW, db, dX = np.zeros_like(W), np.zeros(W.shape[1]), np.zeros_like(X)
    dh = dh_last.copy()
    for t in reversed(range(T)):
        dh = dh + dH[:, t]
        m_t = mask[:, t:t + 1]
        h_prev = Hs[:, t - 1] if t > 0 else h0
        z, r, n, x = Z[:, t], R[:, t], N[:, t], X[:, t]
        d = dh * m_t
        dz = d * (h_prev - n)
        dn = d * (1.0 - z)
        dh_prev = d * z + dh * (1.0 - m_t)
        da_n = dn * (1.0 - n ** 2)
        rh = r * h_prev
        dW[:D, 2 * H:] += x.T @ da_n
        dW[D:, 2 * H:] += rh.T @ da_n
        db[2 * H:] += da_n.sum(axis=0)
        d_rh = da_n @ W[D:, 2 * H:].T
        dr = d_rh * h_prev
        dh_prev += d_rh * r
        da_zr = np.concatenate([dz * z * (1.0 - z), dr * r * (1.0 - r)], axis=1)
        zin = np.concatenate([x, h_prev], axis=1)
        dW[:, :2 * H] += zin.T @ da_zr
        db[:2 * H] += da_zr.sum(axis=0)
        dzin = da_zr @ W[:, :2 * H].T
        dX[:, t] = dzin[:, :D] + da_n @ W[:D, 2 * H:].T
        dh = dh_prev + dzin[:, D:]
    return dW, db, dX, dh
```

The subtle part: the old state reaches the candidate only through r · h, so its gradient from that route is d(r·h) · r, and the reset gate's is d(r·h) · h.

ACTUAL OUTPUT

```
(a) gru_backward on its own:
  array  entries  max relative error  max |difference|    pass
      W       45            1.44e-08          1.27e-10    True
      b        9            2.03e-09          6.50e-11    True
      X       40            1.18e-09          5.15e-11    True
     h0       12            4.05e-10          5.47e-11    True
every entry passes: True;  gradient at padded inputs exactly zero: True
```

```
(b) through GRUClassifier:
  array  entries  max relative error  max |difference|    pass
      W       45            9.97e-08          1.91e-11    True
      b        9            2.06e-09          1.16e-11    True
    Why        6            3.05e-10          9.63e-12    True
     by        2            4.69e-12          8.81e-13    True
every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes, on its own (largest relative error 1.44e-08) and through the classifier (9.97e-08). The gradient at padded inputs is exactly zero.

WHY CHOOSE THIS?

A shorter, checkable backward pass than the LSTM's.

WHY NOT THIS? (TRADEOFFS)

Still easy to get wrong by hand; the r · h route is the one a first draft usually misses.

3. The direct route: the update gate plays the forget gate's part

DEFINITION — start here, no prior knowledge assumed

Going back one step, h = z · h_{prev} + … sends dh · z straight to h_{prev}. With the gates and candidate cut off from h, that is the only route back, so the gradient at h_0 must be the gradient at h_T times the product of the update gates.

REAL-WORLD SCENARIO

The relay with the written note again, but now the note *is* the runner's own summary: each runner keeps a share z of it and rewrites the rest.

MATHEMATICS

gates independent of h: $dL/dh_0 = dL/dh_T \cdot z_1 \cdot \ldots \cdot z_T$ *exactly*

SPECIFICATION

Setup	16 units, 200 sequences of 30 N(0,1) inputs, keep-bias 2, random weights (seed 3), a random gradient placed on h_T. Same setup as Day 65's Section 3.

Why this test

It confirms the GRU has the same direct route as the LSTM, and shows how much travels by the other routes.

IMPLEMENTATION

```
for label, cut in [("gates cut off from h", True), ("normal GRU", False)]:
    Wc = W.copy()
    if cut:
        Wc[1:, :] = 0.0
    _, cache = gru_forward(X, np.zeros((m, H)), Wc, b)
    dh0 = gru_backward(np.zeros((m, T, H)), dh_T, cache)[3]
    direct = dh_T * np.prod(cache["Z"], axis=1)
```

ACTUAL OUTPUT

```
gates cut off from h: ||dL/dh_0|| = 1.184025e+00,  direct route alone = 1.184025e+00,  max
    |difference| = 6.94e-17
normal GRU: ||dL/dh_0|| = 3.077409e+00,  direct route alone = 1.203778e+00,  max |difference| =
```

```
    2.17e-01
Actual output from this lesson's run.
```

What it means

With no indirect route the backward pass equals the product of update gates (difference 6.94e-17). In the normal GRU the total is 3.077409e+00 against 1.203778e+00 for the direct route: the other routes add about 2.6 times as much. In Day 65's identical LSTM setup they added about 47 times (6.125373e+01 against 1.313349e+00). The GRU's state is a blend that stays between –1 and 1; the LSTM's cell state can grow, and so can the gradient through it.

WHY CHOOSE THIS?

The same bounded direct route as the LSTM, with far less amplification on the other routes.

WHY NOT THIS? (TRADEOFFS)

Less amplification also means less gradient reaching early steps when the direct route is weak (Section 4).

4. How much gradient reaches the first input, before training

DEFINITION — start here, no prior knowledge assumed

Day 65's measurement for the GRU: $||dL/dx_0|| / ||dL/dx_{T-1}||$ on Day 59's sign task, for an untrained network, by the update gate's bias.

REAL-WORLD SCENARIO

The same essay comments as Day 65: do they still reach page 1? Now with a different kind of teacher.

SPECIFICATION

Setup	GRUClassifier, 16 units, seed 7, untrained; Day 59's 400 test sequences at 20, 40, 100 steps; keep-bias 0, 1, 2.

Why this test

On Day 65 this number predicted which LSTM settings could learn. Here it predicts the GRU's.

IMPLEMENTATION

```
net = GRUClassifier(1, 16, 2, seed=7, keep_bias=kb)
net.forward(X[:, :, None]); net.gradients(y)
norms = np.linalg.norm(net.dX[:, :, 0], axis=0)
table[(T, kb)] = norms[0] / norms[-1]
```

ACTUAL OUTPUT

```
    T   keep bias 0.0   keep bias 1.0   keep bias 2.0
   20       4.956e-04       1.348e-02       1.390e-01
   40       1.221e-06       4.270e-04       2.383e-02
  100       1.534e-14       4.752e-08       2.580e-04
Actual output from this lesson's run.
```

What it means

At the same bias the GRU passes back much less than Day 65's LSTM. At keep-bias 2: 1.390e-01, 2.383e-02, 2.580e-04 at 20, 40, 100 steps, against the LSTM's 6.973e-01, 1.524e+00, 9.932e+00. The difference is Section 3's: the LSTM's extra routes amplify, the GRU's barely do. At keep-bias 0 both are tiny (1.534e-14 for the GRU at 100 steps).

WHY CHOOSE THIS?

The GRU's gradient is closer to the plain direct-route value, so it is predictable and does not blow up.

WHY NOT THIS? (TRADEOFFS)

With nothing amplifying it, a long sequence needs the keep gate very close to 1 at the start.

5. Training on Day 65's grid

DEFINITION — start here, no prior knowledge assumed

Exactly Day 65's experiment with the LSTM swapped for the GRU: Day 59's sign task (label = sign of the first input), lengths 20/40/100, keep-bias 0/1/2, learning rate 0.05/0.2, seeds 7/8/9, 150 epochs, clipping at 1.0. A run has learned if its final test accuracy is at least 0.9.

REAL-WORLD SCENARIO

Two students take the same exam with the same study time. One finishes more questions. That does not yet say who understands more: maybe the other just works more slowly. Section 5b gives the second student more time.

SPECIFICATION

Grid	54 runs, identical to Day 65 except for the cell.
Reference	Day 65's LSTM learned 0 of 18 runs at bias 0, 10 of 18 at bias 1, 16 of 18 at bias 2.

Why this test

A like-for-like comparison: same data, same settings, same seeds, only the cell differs.

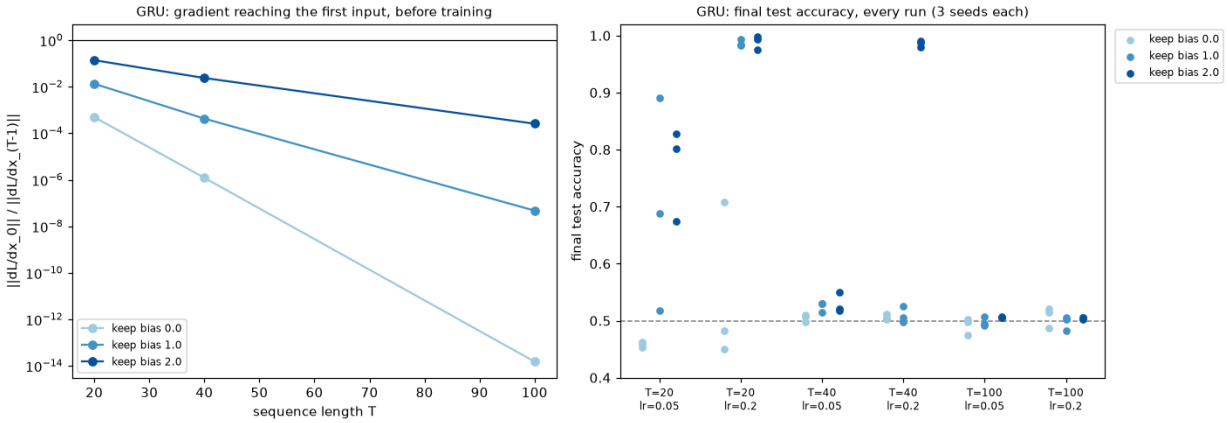
IMPLEMENTATION

```
def train_runs(T, kb, lr, seeds=(7, 8, 9), epochs=150):
    X_tr, y_tr = make_sign_task(600, T, seed=42)
    X_te, y_te = make_sign_task(400, T, seed=999)
    X_tr, X_te = X_tr[:, :, None], X_te[:, :, None]
    runs = []
    for seed in seeds:
        net = GRUClassifier(1, 16, 2, seed=seed, keep_bias=kb, lr=lr, clip=1.0)
        rng = np.random.RandomState(seed)
        curve = []
        for epoch in range(epochs):
            order = rng.permutation(600)
            for start in range(0, 600, 30):
                idx = order[start:start + 30]
                net.train_step(X_tr[idx], y_tr[idx])
            if epoch % 10 == 0 or epoch == epochs - 1:
                curve.append(net.accuracy(X_te, y_te))
        runs.append((max(curve), curve[-1]))
    return runs
```

ACTUAL OUTPUT

```
   T  k-bias    lr   peak / final test accuracy, seeds 7, 8, 9    final >= 0.9
  20     0.0  0.05   0.552/0.453  0.520/0.460  0.525/0.463               0 of 3
  20     0.0   0.2   0.527/0.482  0.787/0.708  0.530/0.450               0 of 3
  20     1.0  0.05   0.532/0.517  0.688/0.688  0.890/0.890               0 of 3
  20     1.0   0.2   0.985/0.985  0.983/0.983  0.993/0.993               3 of 3
  20     2.0  0.05   0.675/0.675  0.828/0.828  0.805/0.802               0 of 3
  20     2.0   0.2   0.998/0.998  0.993/0.993  0.998/0.975               3 of 3
  40     0.0  0.05   0.507/0.497  0.535/0.510  0.527/0.507               0 of 3
  40     0.0   0.2   0.535/0.505  0.540/0.512  0.540/0.502               0 of 3
  40     1.0  0.05   0.537/0.530  0.537/0.530  0.530/0.515               0 of 3
  40     1.0   0.2   0.522/0.505  0.525/0.497  0.530/0.525               0 of 3
  40     2.0  0.05   0.520/0.517  0.525/0.520  0.560/0.550               0 of 3
```

```
  40     2.0   0.2   0.988/0.988  0.990/0.990  0.985/0.980              3 of 3
 100     0.0  0.05   0.545/0.475  0.527/0.497  0.520/0.502              0 of 3
 100     0.0   0.2   0.527/0.487  0.527/0.520  0.530/0.515              0 of 3
 100     1.0  0.05   0.565/0.492  0.520/0.507  0.512/0.495              0 of 3
 100     1.0   0.2   0.527/0.482  0.527/0.502  0.522/0.505              0 of 3
 100     2.0  0.05   0.552/0.505  0.525/0.507  0.517/0.505              0 of 3
 100     2.0   0.2   0.522/0.505  0.525/0.505  0.522/0.502              0 of 3
runs that learned, out of 18 per keep-bias: bias 0.0: 0, bias 1.0: 3, bias 2.0: 6
Actual output from this lesson's run.
```



Left: the GRU's gradient reach before training. Right: final test accuracy of all 54 GRU runs; each dot is one seed.

What it means

Fewer runs learned than with the LSTM: 0, 3 and 6 of 18 at keep-bias 0, 1, 2 (LSTM: 0, 10, 16). Keep-bias 0 fails everywhere for both cells.

Where the GRU learns: only with the larger learning rate, 0.2. At 20 steps it learns every run with keep-bias 1 or 2; at 40 steps every run with keep-bias 2. At Day 60's learning rate 0.05 it learns nothing within 150 epochs, and at 100 steps it learns nothing at all on this grid.

WHY CHOOSE THIS?

Like-for-like, on this task and budget, the LSTM is the more reliable cell.

WHY NOT THIS? (TRADEOFFS)

One task, one budget: a slower learner and a weaker learner look the same here. Section 5b separates them.

5b. Follow-up: slower, or unable?

DEFINITION — start here, no prior knowledge assumed

A **follow-up** changes one setting at a time where the first experiment failed, to see whether the failure is about the method or about the setting. Also measured: the share of the candidate the GRU can write at the first step, 1 – z, which is tied to how much it keeps.

REAL-WORLD SCENARIO

Back to the bathtub: the LSTM has a separate plug (forget gate) and tap (input gate). The GRU has one lever: pushing it towards “keep the water in” also turns the tap off. A lever set to keep 98% of the water lets in only 2% of new water.

MATHEMATICS

GRU: keep = z, write = 1 – z tied; at keep-bias b: z ≈ sigmoid(b), 1 – z ≈ sigmoid(–b)

LSTM: keep = f, write = i separate gates; the forget bias does not change i

SPECIFICATION

Settings	Five, one change at a time from the grid, each with seeds 7, 8, 9: 40 steps at lr 0.05 with 450 epochs; keep-bias 3 and 4 at lr 0.2, at 40 and at 100 steps.
Gate table	At the first step of the 40-step test sequences, the seed-7 GRU's mean z and 1 – z, against an LSTM's f and i at the same bias.

Why this test

Section 5 cannot tell “the GRU learns this more slowly” from “the GRU cannot learn this”. Each row below tests one of those for one failing setting.

IMPLEMENTATION

```
for label, T, kb, lr, epochs in [("T=40, keep-bias 2, lr 0.05, 450 epochs", 40, 2.0, 0.05, 450),
                                 ("T=40, keep-bias 3, lr 0.2", 40, 3.0, 0.2, 150),
                                 ("T=40, keep-bias 4, lr 0.2", 40, 4.0, 0.2, 150),
                                 ("T=100, keep-bias 3, lr 0.2", 100, 3.0, 0.2, 150),
                                 ("T=100, keep-bias 4, lr 0.2", 100, 4.0, 0.2, 150)]:
    runs = train_runs(T, kb, lr, epochs=epochs)
    rows[label] = runs
    cells = "  ".join(f"{p:.3f}/{f:.3f}" for p, f in runs)
    print(f"{label:<44}{cells:<40}{sum(f >= 0.9 for _, f in runs):>9} of 3")
```

ACTUAL OUTPUT

```
 keep bias  GRU keeps z  GRU writes 1 - z  LSTM keeps f  LSTM writes i
       2.0        0.879             0.121         0.881          0.500
       3.0        0.952             0.048         0.953          0.500
       4.0        0.982             0.018         0.982          0.500
(the LSTM columns are its gates at a raw score of the bias alone: f = sigmoid(bias), i =
    sigmoid(0))
```

ACTUAL OUTPUT

```
setting                                  peak / final, seeds 7, 8, 9              final >= 0.9
T=40, keep-bias 2, lr 0.05, 450 epochs   0.693/0.693  0.985/0.985  0.988/0.988           2 of 3
T=40, keep-bias 3, lr 0.2                0.990/0.983  0.995/0.995  0.993/0.970           3 of 3
T=40, keep-bias 4, lr 0.2                0.603/0.588  0.635/0.635  0.620/0.615           0 of 3
T=100, keep-bias 3, lr 0.2               0.525/0.502  0.525/0.517  0.527/0.507           0 of 3
T=100, keep-bias 4, lr 0.2               0.560/0.525  0.568/0.525  0.552/0.525           0 of 3
Actual output from this lesson's run.
```

What it means

At 40 steps the lr-0.05 failure was mostly speed: with three times the epochs, 2 of 3 runs learn (peak / final per seed: 0.693/0.693 0.985/0.985 0.988/0.988). The GRU learns this task more slowly than the LSTM at the same learning rate, not never.

The tied gate is a real cost. At 40 steps keep-bias 3 learns 3 of 3, but keep-bias 4 learns 0 of 3. At bias 4 the GRU keeps 0.982 of its state at the first step and so can write only 0.018 of the candidate, which is where the first input has to go in. The LSTM at the same forget bias still writes with its separate input gate at 0.500. On Day 63 a high forget bias was the LSTM's best setting; for the GRU the same move blocks the input it is supposed to remember.

100 steps: keep-bias 3 learns 0 of 3 and keep-bias 4 0 of 3 runs. Within these settings and 150 epochs, the GRU did not match the LSTM's 4 of 6 at 100 steps; whether a longer budget would close that gap was not

tested here.

WHY CHOOSE THIS?

Fewer parameters, and with a suitable learning rate it learns the 20- and 40-step tasks as well as the LSTM.

WHY NOT THIS? (TRADEOFFS)

Its keep and write shares are one gate, so the trick that made the LSTM robust (a high forget bias) helps the GRU only up to a point and then hurts.

6. What one step costs

DEFINITION — start here, no prior knowledge assumed

Time per step, measured: 20,000 single steps of each cell with input 32, hidden 32, batch 32, with the matrix library on one thread.

REAL-WORLD SCENARIO

A shop with three staff against one with four: fewer salaries, though not always proportionally less work done.

IMPLEMENTATION

```
t_gru = best(lambda: gru_step_forward(x, h, Wg, bg))
t_lstm = best(lambda: lstm_step_forward(x, h, c, Wl, bl))
```

ACTUAL OUTPUT

```
20,000 single steps, input 32, hidden 32, batch 32, matrix library on one thread, best of 5
    timings. Timings vary by machine.
GRU 0.700 s, LSTM 0.979 s for 20,000 steps; GRU / LSTM time ratio 0.72
Actual output from this lesson's run.
```

What it means

In this run one GRU step took 0.72 times as long as one LSTM step. Weights are three quarters; time saved depends on the machine and the sizes.

WHY CHOOSE THIS?

Cheaper per step and smaller.

WHY NOT THIS? (TRADEOFFS)

The saving is modest and does not make up for a cell that fails to learn the task.

Interview preparation — technical questions

Q: Write the GRU equations.

A: z = sigmoid([x, h] Wz + bz); r = sigmoid([x, h] Wr + br); n = tanh(x Wxn + (r * h) Whn + bn); h_new = z * h + (1 - z) * n. Check which convention a library uses: some swap z and 1 - z.

Q: How many parameters does a GRU layer have?

A: 3 (D + H + 1) H: three quarters of an LSTM of the same size.

Q: Why can the candidate's hidden weights not be stacked with the gates'?

A: The candidate multiplies r * h, not h, so r must be computed first; the two gates can share one product, the candidate needs its own.

Q: What does the GRU's backward pass send to the previous state?

A: dh * z directly, plus d(r * h) * r through the candidate, plus the gates' share through [x, h].

Q: Should the GRU's update-gate bias be set high like an LSTM's forget bias?

A: Moderately. Here keep-bias 3 learned 40 steps in every run but keep-bias 4 failed, because 1 - z, the write share, fell to about 0.02.

Interview preparation — theoretical questions

Q: How does the GRU differ from the LSTM?

A: One state instead of two, no output gate, and keep/write tied in one update gate instead of separate forget and input gates.

Q: Which is better?

A: Neither in general. On this task and budget the LSTM learned more runs; the GRU matched it at 20 and 40 steps with a larger learning rate and a moderate keep-bias, and is cheaper per step.

Q: Why does the LSTM's gradient reach early steps more strongly at the same bias?

A: Its cell state is an unbounded sum, so the indirect routes amplify (47x the direct route in Day 65's setup); the GRU's state is a bounded blend (2.6x).

Q: What is the reset gate for?

A: It lets the candidate ignore the old state when computing what to write, for example when the input starts something new.

Key takeaway and what's next

> **KEY TAKEAWAY**
> The GRU keeps the LSTM's direct route (dh * z is the update gate's version of dc * f), checks out to rounding error, and uses three quarters of the weights. Like-for-like on Day 65's grid it learned fewer runs; given more epochs or a larger learning rate it learned the 40-step task too. Its real cost is the tied gate: keeping more always means writing less, so a high keep-bias, the LSTM's best setting, stops the GRU from taking in the very input it has to remember.

- GRU: one state, update gate z (keep), reset gate r, candidate n; h_new = z * h + (1 - z) * n.
- Weights: 3 (D + H + 1) H; one GRU step is cheaper than one LSTM step.
- Direct route dh_prev = dh * z, exactly the product of update gates when nothing else depends on h.
- The GRU's indirect routes amplify much less than the LSTM's, so less gradient reaches early steps.
- Keep-bias: 0 fails, 3 learned 40 steps every time, 4 failed because 1 - z fell to 0.018.
- Separate slow from unable before ranking two methods.

Suggested watch order, if starting from zero

- 1. Simple Explanation of GRU (Gated Recurrent Units) | Deep Learning Tutorial 37 (codebasics) — the GRU in plain terms, next to the LSTM.
- 2. Gated Recurrent Unit (GRU) Equations Explained (DataMListic) — the equations one at a time.

Day 67 preview

- **RNN vs LSTM vs GRU on real data.** All three cells now have checked gradients. Day 67 trains them on Week 9's part-of-speech tagging task, which needs mostly short-range context, to see whether today's long-memory differences matter on a real problem.
- Today's exercise script, `day66_exercise.py`, is the GRU step on its own.